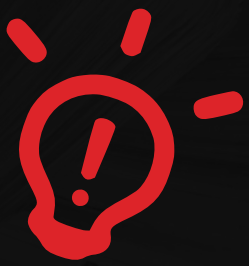


COMPUTER ORGANISATION AND DESIGN

Understanding the foundation of modern computer systems

Table of Contents

UNIT 1 – Basics of Digital Electronics

1. COMBINATIONAL CIRCUITS
    - 1.1 HALF ADDER & FULL ADDER
    - 1.2 HALF SUBTRACTOR & FULL SUBTRACTOR
    - 1.3 MULTIPLEXER (MUX) & DEMULTIPLEXER (DEMUX)
    - 1.4 DECODER AND ENCODER
2. SEQUENTIAL CIRCUITS
    - 2.1 LATCHES (SR, D)
    - 2.2 FLIP-FLOPS (SR,D)
    - 2.3 SHIFT REGISTERS

UNIT 2 - Computer arithmetic

1. DATA REPRESENTATION
    - 1.1 INTEGER ADDITION AND SUBTRACTION
    - 1.2 RIPPLE CARRY & CARRY LOOK-AHEAD ADDERS
    - 1.3 MULTIPLICATION ALGORITHMS (BOOTH'S ALGORITHM)
    - 1.4 DIVISION ALGORITHMS
    - 1.5 FIXED AND FLOATING-POINT

UNIT 3 – Basic Processing Module

1. FUNDAMENTAL CONCEPTS
    - 1.1 EXECUTION OF A COMPLETE INSTRUCTION (FETCH-DECODE-EXECUTE CYCLE)
2. CPU ORGANIZATION
    - 2.1 MULTIPLE BUS ORGANIZATION
3. CONTROL UNIT
    - 3.1 HARDWIRED CONTROL
    - 3.2 MICROPROGRAMMED CONTROL
4. PIPELINING AND HAZARDS
    - 4.1 DATA HAZARDS
    - 4.2 INSTRUCTION (CONTROL) HAZARDS

UNIT 1
Basics of Digital Electronics

Digital electronics operate on discrete signals, typically represented as binary values (0 and 1). The fundamental building blocks are logic gates (AND, OR, NOT, etc.), which are combined to create more complex circuits.

Combinational Circuits

Combinational circuits are very well known components in digital electronics which can provide output instantly based on the current input. Unlike sequential circuits, a combinational circuit listens for input signal and and generates output no matter what is the past input or state as it has no feedback or memory component. It only cares about present input and state.

They are specially designed using multiple interconnected logic gates such that the output will be generated by computing the logical combinations of the present input only. No clock pulse is present here. Moreover, no previously stored value or state is taken into consideration here. The output is independent of previous states.

Features of Combinational Circuits

- In this output depends only upon present input.
- It's Speed is fast.
- Easy designed.
- There is no feedback between input and output.
- It is time independent.
- Elementary building blocks are Logic gates.
- Used for both arithmetic and boolean operations.
- Combinational circuits don't have the capability to store any state.

Figure - Block diagram of Combinational circuit

Applications of Combinational Circuits

In modern technologies, combinational circuits are widely used for its simple functionality and ability to give instant output. Some of the applications are discussed below:

- Arithmetic and Logic Units (ALUs): As combinational circuits are capable of performing various mathematical tasks like arithmetic and logical operations so, these circuits are used in calculators and processors as a fundamental component of Arithmetic and Logic Units (ALUs).
- Data Encryption and Decryption: In information protection fields, combinational circuit are used for

- Data Encryption and decryption of data for secure communication. Encryption/decryption algorithms are also a complex mathematical formula which can be performed by combinational circuits.
- Data Multiplexing and Demultiplexing: It is just a practical implementation of multiplexer and demultiplexer. By using it we can optimize network bandwidth by effectively reducing network traffic as combinational circuit allows to transmit multiple data signals over a single communication channel.
- Traffic Light Control: In traffic lights control mechanism, combinational circuits are used to instantly determine the timing and sequence of traffic light changes based on the inputs of timers and sensors.

Types of Combinational Circuits

- Adders and Subtractors
- Multiplexers and Demultiplexers
- Encoders and Decoders

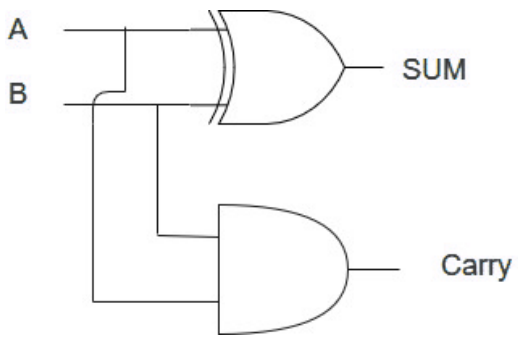
Half Adder

A half adder is a combinational logic circuit that performs binary addition of two single-bit inputs, A and B, producing two outputs: SUM and CARRY. The SUM output which is the least significant bit (LSB) is obtained using an XOR gate while the CARRY output which is the most significant bit (MSB) is generated using an AND gate. The half adder is a fundamental building block for more complex adder circuits, such as full adders and multi-bit adders.

Truth Table of Half Adder

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Implementation of Half Adder



Note: Half adder has only two inputs and there is no provision to add a carry coming from the lower order bits when multi addition is performed.

Full Adder

Full Adder is a combinational circuit that adds three inputs and produces two outputs. The first two inputs are A and B and the third input is an input carry as C-IN. The output carry is designated as C-OUT and the normal output is designated as S which is SUM.

- The C-OUT is also known as the majority 1's detector, whose output goes high when more than one input is high.
- A full adder logic is designed in such a manner that can take eight inputs together to create a byte-wide adder and cascade the carry bit from one adder to another.
- We use a full adder because when a carry-in bit is available, another 1-bit adder must be used since a 1-bit half-adder does not take a carry-in bit.
- A 1-bit full adder adds three operands and generates 2-bit results.

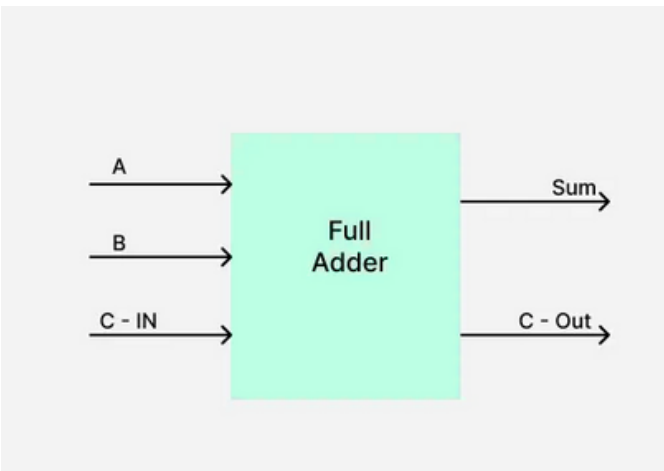
Full Adder Truth Table

A Full Adder takes three binary inputs:

- A (first bit)
- B (second bit)
- C-IN (carry input)

And it produces two outputs:

- Sum (S)
- Carry Out (C-OUT)



Here's the truth table for the full adder:

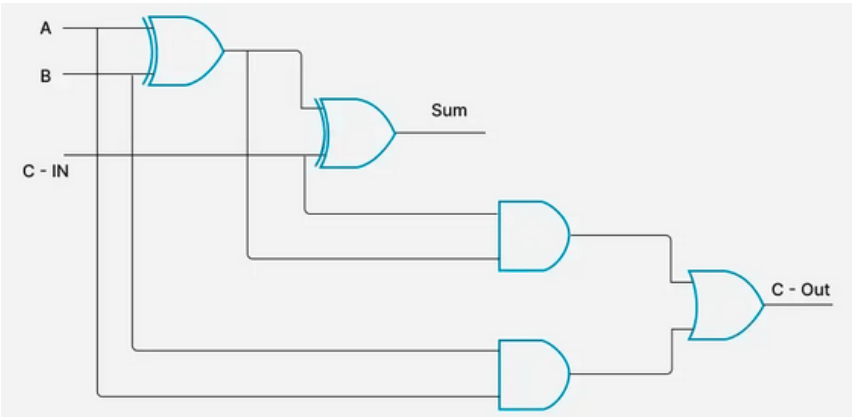
INPUT			OUTPUT	
A	**B**	**C-IN**	**Sum**	**C-OUT**
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Logic Circuit of Full Adder

To implement a Full Adder using basic logic gates:

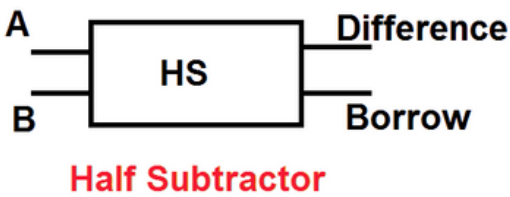
1. Sum (S) is implemented using XOR gates:
   - Use two XOR gates:
     - First XOR gate: A ⊕ B
     - Second XOR gate: (A ⊕ B) ⊕ C-IN to get the final sum S.
2. Carry (C-Out) is implemented using XOR,AND and OR gates:
   - First AND gate: This gate calculates A AND B.
   - Second AND gate: This gate calculates C-IN AND (A⊕ B).

Finally, the two outputs from the AND gates are combined using an OR gate to generate the final C-OUT output.



Half subtracter

A half subtractor is a digital logic circuit that performs the binary subtraction of two single-bit binary numbers. It has two inputs, A and B, and two outputs, Difference and Borrow. The Difference output represents the result of subtracting B from A, while the Borrow output indicates whether a borrow is needed when A is smaller than B.



Half Subtractor

Truth Table of Half Subtractor

A	B	Diff	Borrow
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

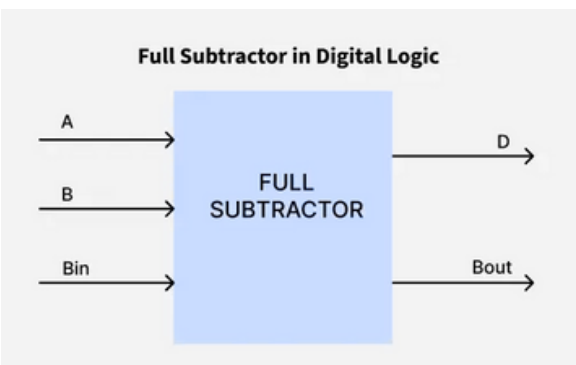
Full Subtractor

A Full Subtractor is a combinational circuit used to perform binary subtraction. It has three inputs:

- A (Minuend)
- B (Subtrahend)
- B-IN (Borrow-in from the previous stage)

It produces two outputs:

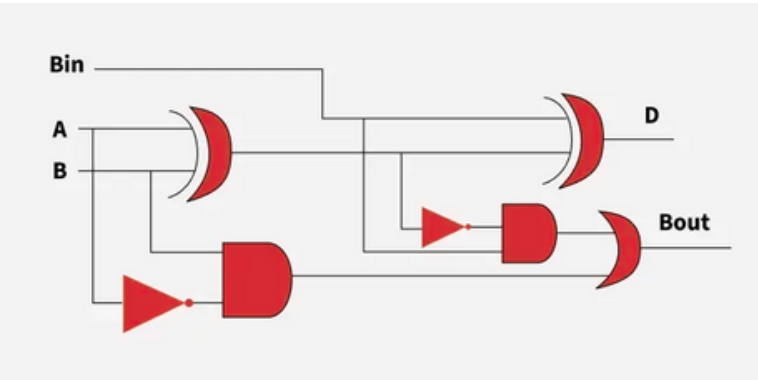
- Difference (D): The result of the subtraction.
- Borrow-out (B-OUT): Indicates if a borrow is needed for the next stage.



Here's the truth table for the full subtracter:

Input			Output	
A	**B**	**Bin**	**D**	**Bout**
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

Logic Circuit for Full Subtractor



Multiplexers in Digital Logic

- A multiplexer is a combinational circuit that has many data inputs and a single output, depending on control or select inputs. For N input lines, $\log_2(N)$ selection lines are required, or equivalently, for 2^n input lines, n selection lines are needed. Multiplexers are also known as "N-to-1 selectors," parallel-to-serial converters, many-to-one circuits, and universal logic circuits. They are mainly used to increase the amount of data that can be sent over a network within a certain amount of time and bandwidth .

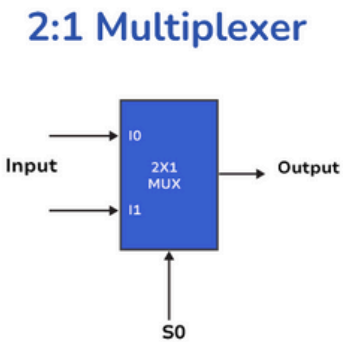
Types of Mux

The Mux can be of different types based on input but in this article we will go through two major types of mux which are

- 2x1 Mux
- 4x1 Mux

2x1 Multiplexer

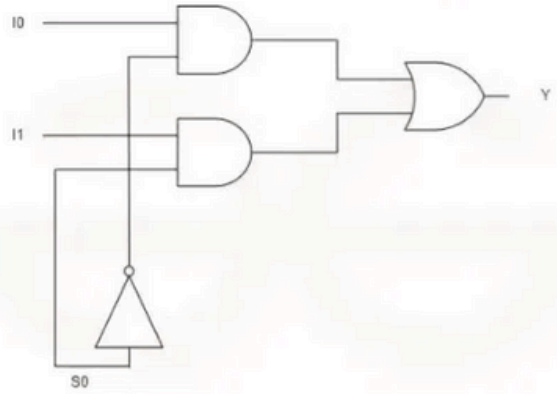
The 2x1 is a fundamental circuit which is also known 2-to-1 multiplexer that are used to choose one signal from two inputs and transmits it to the output. The 2x1 mux has two input lines, one output line, and a single selection line.



Truth Table

S_0	I_0	I_1	Y
0	0	X	0
0	1	X	1
1	X	0	0
1	X	1	1

Circuit Diagram of 2x1 Multiplexers



4x1 Multiplexer

The 4x1 Multiplexer which is also known as the 4-to-1 multiplexer. It is a multiplexer that has 4 inputs and a single output. The Output is selected as one of the 4 inputs which is based on the selection inputs. The number of the Selection lines will depend on the number of the input which is determined by the equation log2nlog2n ,In 4x1 Mux the selection lines can be determined as log4=2log4=2 ,slo two selections are needed.

The output of the multiplexer is determined by the binary value of the selection lines

- When S1S0=00, the input I0 is selected.
- When S1S0=01, the input I1 is selected.
- When S1S0=10, the input I2 is selected.
- When S1S0=11, the input I3 is selected.

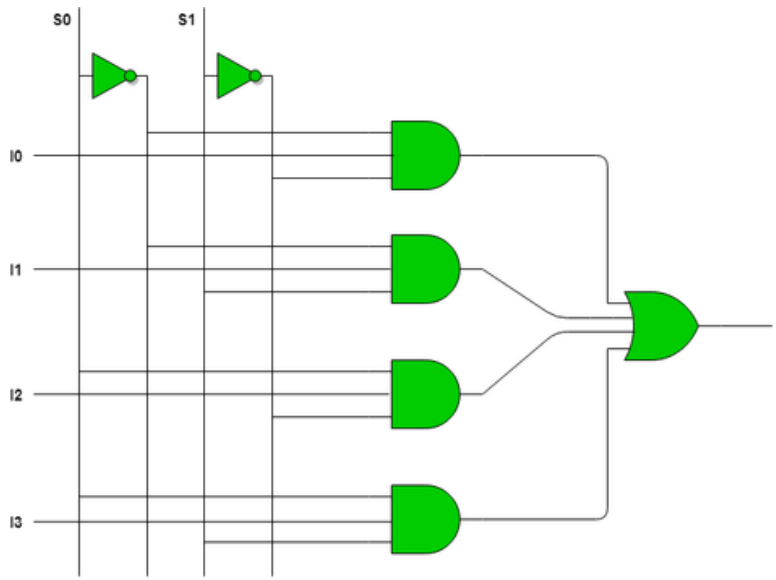
Truth Table

S0	S1	Y
0	0	I0
0	1	I1
1	0	I2
1	1	I3

So, final equation,

$Y = S0'.S1'.I0 + S0'.S1.I1 + S0.S1'.I2 + S0.S1.I3$

Circuit Diagram of 4x1 Multiplexers

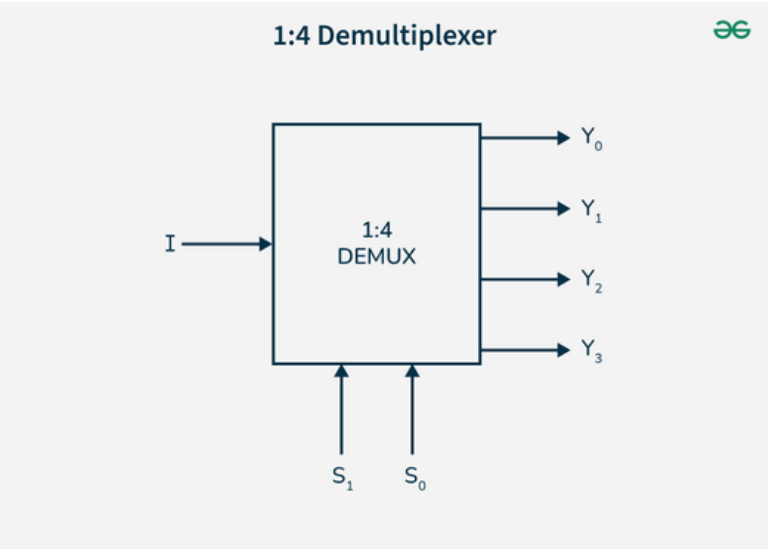


Demultiplexers in Digital Logic

- The DEMUX is a digital information processor. It takes input from one source and also converts the data to transmit towards various sources. The demultiplexer has one data input line. The demultiplexer has several control lines (also known as select lines). These lines determine to which output the input data should be sent. The number of control lines determines the number of output lines.

1x4 Demultiplexer

A 1x4 DEMUX has only one input which is denoted as I. There are two selection lines i.e. S1 and S0. At last, the DEMUX has output lines including Y3, Y2, Y1 &Y0.



The truth table of the 1x4 DEMUX:

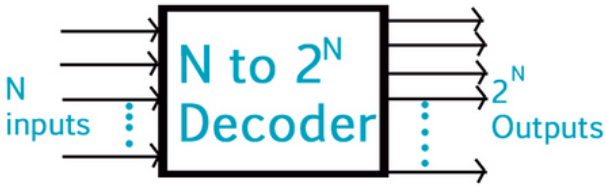
Selection		Outputs			
S1	**S0**	**Y3**	**Y2**	**Y1**	**Y0**
0	0	0	0	0	**I**
0	1	0	0	**I**	0
1	0	0	**I**	0	0
1	1	**I**	0	0	0

Binary Decoder in Digital Logic

A binary decoder is a digital circuit used to convert binary-coded inputs into a unique set of outputs. It does the opposite of what an encoder does. A decoder takes a binary value (such as 0010) and activates exactly one output line corresponding to that value while all other output lines remain inactive.

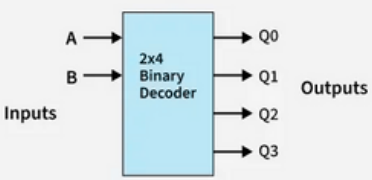
A decoder uses n input lines to generate 2^n unique outputs. Each binary input combination maps to exactly one output being active. For example:

- A 2-bit decoder has 2 inputs and 4 outputs ($2^2 = 4$).
- A 4-bit decoder has 4 inputs and 16 outputs ($2^4 = 16$).



2-to-4 Binary Decoder

A 2-to-4 decoder is a combinational digital circuit that takes 2 binary inputs and activates one of four outputs, based on the input combination. Only one output is active at a time, and all others remain low.
The circuit decodes the 2-bit binary input into one of the four output lines.

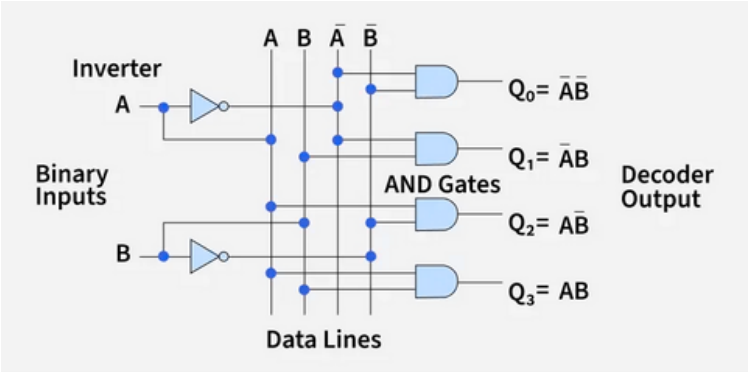


A	B	Q0	Q1	Q2	Q3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

Given binary inputs A and B, the outputs are:

- Q0 = A'B' (when A = 0, B = 0)
- Q1 = A'B (when A = 0, B = 1)
- Q2 = AB' (when A = 1, B = 0)
- Q3 = AB (when A = 1, B = 1)

Note: Only one output is HIGH (1) at a time depending on the combination of A and B.



Enable Input (E):
An additional input called Enable (E) is used to control the output.

- When E = 0 → All outputs are forced to 0 (OFF).
- When E = 1 → Outputs are determined by the values of A and B as per the above logic.

Encoder in Digital Logic

An Encoder is a combinational circuit that performs the reverse operation of a Decoder. It has a maximum of 2^n input lines and 'n' output lines, hence it encodes the information from 2^n inputs into an n-bit code. It will produce a binary code equivalent to the input, which is active High. Therefore, the encoder encodes 2^n input lines with 'n' bits.

Types of Encoders

There are different types of Encoders which are mentioned below.

- 4 to 2 Encoder
- Octal to Binary Encoder (8 to 3 Encoder)
- Decimal to BCD Encoder
- Priority Encoder

4 to 2 Encoder

The 4 to 2 Encoder consists of four inputs Y3, Y2, Y1 & Y0, and two outputs A1 & A0. At any time, only one of these 4 inputs can be '1' in order to get the respective binary code at the output. The figure below shows the logic symbol of the 4 to 2 encoder.

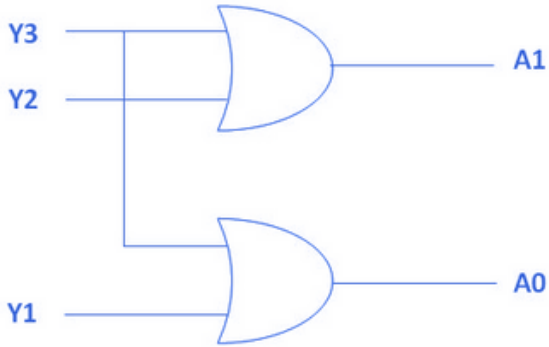
The Truth table of 4 to 2 encoders is as follows.

INPUTS				OUTPUTS	
Y3	**Y2**	**Y1**	**Y0**	**A1**	**A0**
0	0	0	1	0	0
0	0	1	0	0	1
0	1	0	0	1	0
1	0	0	0	1	1

Logical expression for A1 and A0:

A1 = Y3 + Y2

A0 = Y3 + Y1



Introduction of Sequential Circuits

Sequential circuits are digital circuits that store and use previous state information to determine their next state. They are commonly used in digital systems to implement state machines, timers, counters, and memory elements and are essential components in digital systems design. Sequential circuits are commonly used in digital systems to implement state machines, timers, counters, and memory elements. The memory elements in sequential circuits can be implemented using flip-flops, which are circuits that store binary values and maintain their state even when the inputs change.

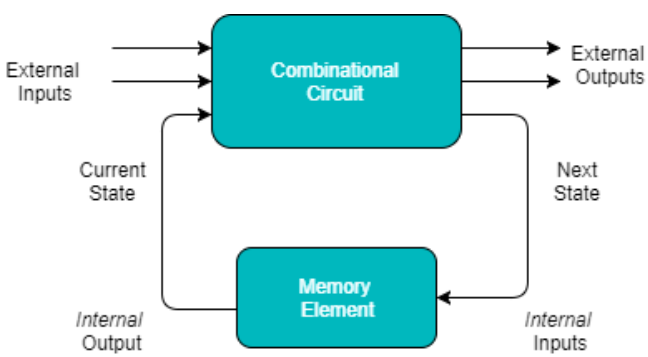


Figure: Sequential Circuit

Types of Sequential Circuits

Asynchronous Sequential Circuit

These circuits do not use a clock signal but uses the pulses of the inputs. These circuits are faster than synchronous sequential circuits because there is clock pulse and change their state immediately when there is a change in the input signal. We use asynchronous sequential circuits when speed of operation is important and independent of internal clock pulse.

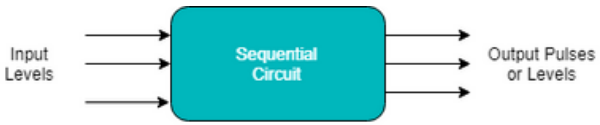


Figure: Asynchronous Sequential Circuit

Synchronous Sequential Circuit

These circuits uses clock signal and level inputs (or pulsed) (with restrictions on pulse width and circuit propagation). The output pulse is the same duration as the clock pulse for the clocked sequential circuits. Since they wait for the next clock pulse to arrive to perform the next operation, so these circuits are bit slower compared to asynchronous. Level output changes state at the start of an input pulse and remains in that until the next input or clock pulse.

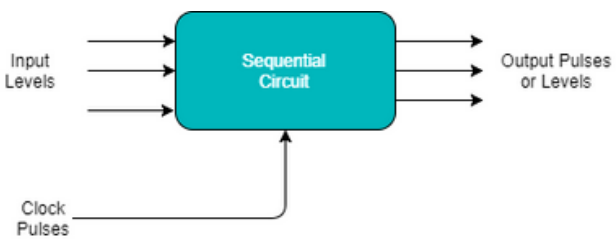


Figure: Synchronous Sequential Circuit

Latches in Digital Logic

Latches are digital circuits that store a single bit of information and hold its value until it is updated by new input signals. They are used in digital systems as temporary storage elements to store binary information.

Types of Latches

- SR Latches
- D Latches & many more.....

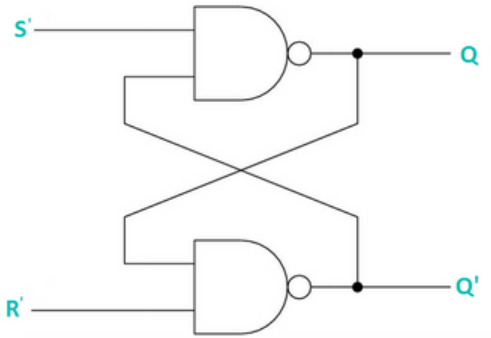
SR Latch

S-R latches i.e., Set-Reset latches are the simplest form of latches and are implemented using two inputs: S (Set) and R (Reset). The S input sets the output to 1, while the R input resets the output to 0. When both S and R inputs are at 1, the latch is said to be in an "undefined" state. They are also known as preset and clear states.

The below table represents the truth table of SR latch.

S	R	Q	Q'
0	0	Latch	Latch
0	1	0	1
1	0	1	0
1	1	0	0

The below logic diagram represents the SR latch using NAND gate.



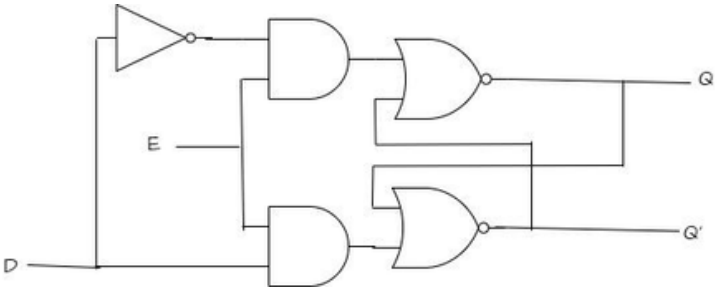
D Latch

D latches are also known as transparent latches and are implemented using two inputs: D (Data) and a clock signal. The output of the latch follows the input at the D terminal as long as the clock signal is high. When the clock signal goes low, the output of the latch is stored and held until the next rising edge of the clock.

The below table represents the truth table of D latch.

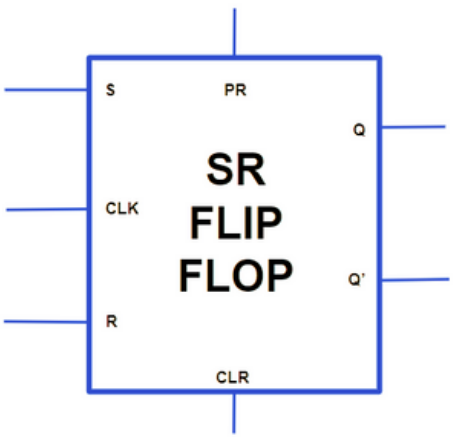
E	D	Q	Q'
0	0	Latch	Latch
0	1	Latch	Latch
1	0	0	1
1	1	1	0

The below logic diagram represents the D latch.

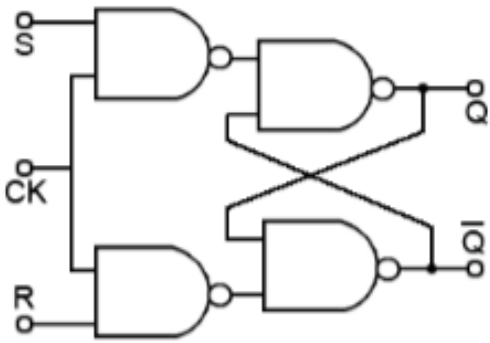


S-R Flip Flop

In the flip flop, with the help of preset and clear when the power is switched ON, the states of the circuit keeps on changing, that is it is uncertain. It may come to set(Q=1) or reset(Q'=0) state. In many applications, it is desired to initially set or reset the flip flop that is the initial state of the flip flop that needs to be assigned. This thing is accomplished by the preset(PR) and the clear(CLR).



Given Below is the Diagram of S-R Flip Flop with its Truth Table



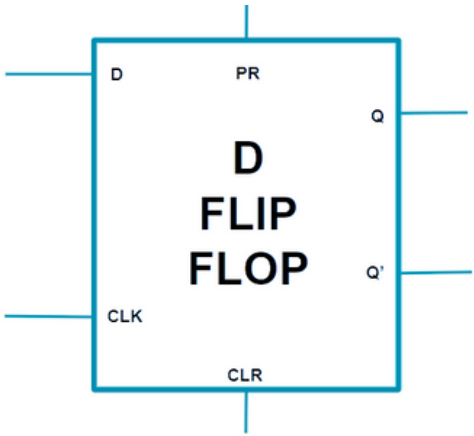
TRUTH TABLE

S	R	Q_N	Q_{N+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	-
1	1	1	-

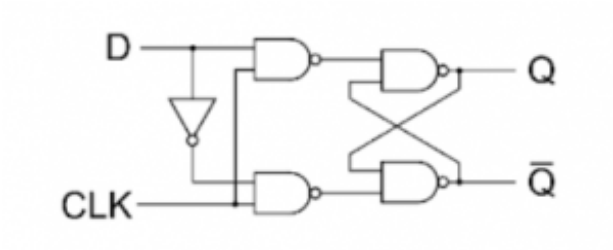
D Flip Flop

The D Flip Flop Consists a single data input(D), a clock input(CLK),and two outputs: Q and Q' (the complement of Q).

Given Below is the Block Diagram of D Flip Flop



Given Below is the Diagram of D Flip Flop with its Truth Table



Q	D	Q(t+1)
0	0	0
0	1	1
1	0	0
1	1	1

Shift Registers in Digital Logic

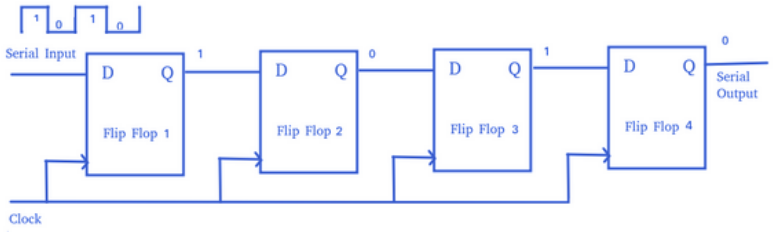
Shift Register is a group of flip flops used to store multiple bits of data. The bits stored in such registers can be made to move within the registers and in/out of the registers by applying clock pulses. An n-bit shift register can be formed by connecting n flip-flops where each flip-flop stores a single bit of data. The registers which will shift the bits to the left are called “Shift left registers”. The registers which will shift the bits to the right are called “Shift right registers”. Shift registers are basically of following types.

Types of Shift Registers

- Serial In Serial Out shift register
- Serial In parallel Out shift register
- Parallel In Serial Out shift register
- Parallel In parallel Out shift register
- Bidirectional Shift Register
- Universal Shift Register
- Shift Register Counter

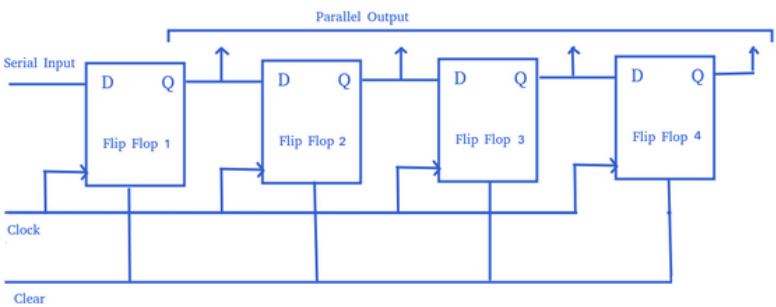
Serial-In Serial-Out Shift Register (SISO)

The shift register, which allows serial input (one bit after the other through a single data line) and produces a serial output is known as a Serial-In Serial-Out shift register. Since there is only one output, the data leaves the shift register one bit at a time in a serial pattern, thus the name Serial-In Serial-Out Shift Register. The logic circuit given below shows a serial-in serial-out shift register. The circuit consists of four D flip-flops which are connected in a serial manner. All these flip-flops are synchronous with each other since the same clock signal is applied to each flip-flop.



Serial-In Parallel-Out Shift Register (SIPO)

The shift register, which allows serial input (one bit after the other through a single data line) and produces a parallel output is known as the Serial-In Parallel-Out shift register. The logic circuit given below shows a serial-in-parallel-out shift register. The circuit consists of four D flip-flops which are connected. The clear (CLR) signal is connected in addition to the clock signal to all 4 flip flops in order to RESET them. The output of the first flip-flop is connected to the input of the next flip flop and so on. All these flip-flops are synchronous with each other since the same clock signal is applied to each flip-flop.



UNIT 2
Computer Arithmetic

Computer arithmetic is the branch of computer science that deals with the representation and manipulation of numerical quantities in a computer system.

Data Representation

Data representation refers to the way data (numbers, characters, instructions, multimedia, etc.) is stored, processed, and interpreted inside a computer system. Since computers only understand binary (0s and 1s), every type of information must be encoded in a binary form before it can be manipulated.

Integer Representation:The Power of 2's Complement

Computers need a way to represent both positive and negative integers. While methods like sign-magnitude exist, the universal standard is 2's complement.
Let's use 4-bit numbers as an example:

- Positive Numbers: These are represented by their standard binary value. The most significant bit (MSB), or the leftmost bit, is 0.
  - Example: +5=01012
- Negative Numbers: This is where 2's complement comes in. To find the 2's complement of a number (i.e., to make it negative):
  a. Step 1: Invert the bits. Flip all the 0s to 1s and all the 1s to 0s. This is called the 1's complement.
  b. Step 2: Add 1 to the result.

Let's find the representation for -5:

1. Start with positive 5: 0101
2. Invert the bits (1's complement): 1010
3. Add 1: 1010 + 1 = 1011 So, −5=10112. Notice the leftmost bit (MSB) is now 1, which signifies a negative number.

Integer Addition

With 2's complement, addition is straightforward, regardless of whether the numbers are positive or negative. You just add the binary numbers together, including the sign bit. Any carry-out from the most significant bit is simply discarded.
Let's look at all four possibilities:

Case 1: Adding Two Positives

Add 5+2:

- 5=0101
- 2=0010

```
 0101   (5)
+ 0010   (2)
-------
 0111   (7)
```

The result is 0111, which is 7. This is correct.

Case 2: Adding a Positive and a Smaller Negative

Add 5+(−2):

- 5=0101
- −2 (Find 2's complement of 0010) → 1101 + 1 = 1110

```
 0101   (5)
+ 1110   (-2)
-------
1 0011   (3) The result is 0011, which is 3. Correct again!
```

A Special Case: Overflow

Overflow occurs when the result of an addition is too large to fit in the number of bits available. The easiest way to detect it is:
Overflow happens if the sign of the two numbers you are adding is the same, but the sign of the result is different.
Example: Add 7+3 in 4 bits.

- 7=0111
- 3=0011

```
 0111   (7)
+ 0011   (3)
-------
 1010   (-6)  <-- WRONG!
```

We added two positives (sign bit 0) but got a result with a sign bit of 1 (a negative). This is an overflow error. The correct answer (10) cannot be represented in 4-bit 2's complement.

Integer Subtraction

This is the most elegant part. The ALU doesn't need a separate "subtractor" circuit. To compute A - B, the computer simply finds the 2's complement of B and adds it to A.
A - B is treated as A + (-B)
Let's calculate 7−5: This is the same as 7+(−5).

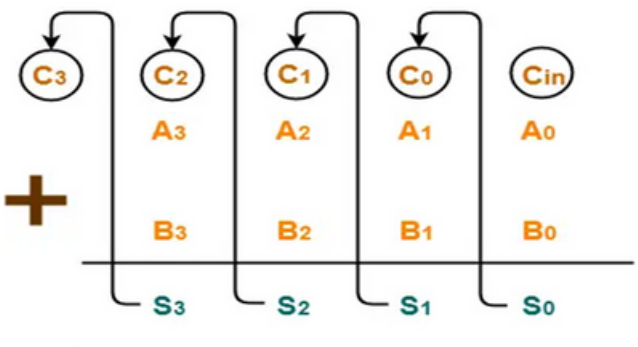
- 7=0111
- −5=1011

```
 0111   (7)
+ 1011   (-5)
-------
1 0010   (2)
```

Discard carry bit
The result is 0010, which is 2.The process works flawlessly by turning every subtraction problem into an addition problem.

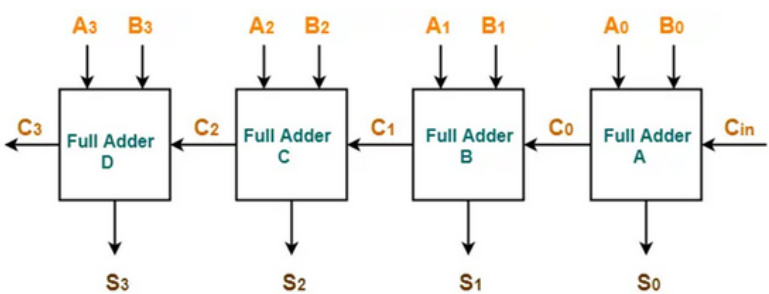
Ripple Carry Adder

- Ripple Carry Adder is a combinational logic circuit.
- It is used for the purpose of adding two n-bit binary numbers.
- It requires n full adders in its circuit for adding two n-bit binary numbers.
- It is also known as n-bit parallel adder.
- 4-bit ripple carry adder is used for the purpose of adding two 4-bit binary numbers.
- In Mathematics, any two 4-bit binary numbers $A_3A_2A_1A_0$ and $B_3B_2B_1B_0$ are added as shown below-



Adding two 4-bit Numbers

Using ripple carry adder, this addition is carried out as shown by the following logic diagram-



4-bit Ripple Carry Adder

- Ripple Carry Adder works in different stages.
- Each full adder takes the carry-in as input and produces carry-out and sum bit as output.
- The carry-out produced by a full adder serves as carry-in for its adjacent most significant full adder.
- When carry-in becomes available to the full adder, it activates the full adder.
- After full adder becomes activated, it comes into operation.

Carry Look Ahead Adder

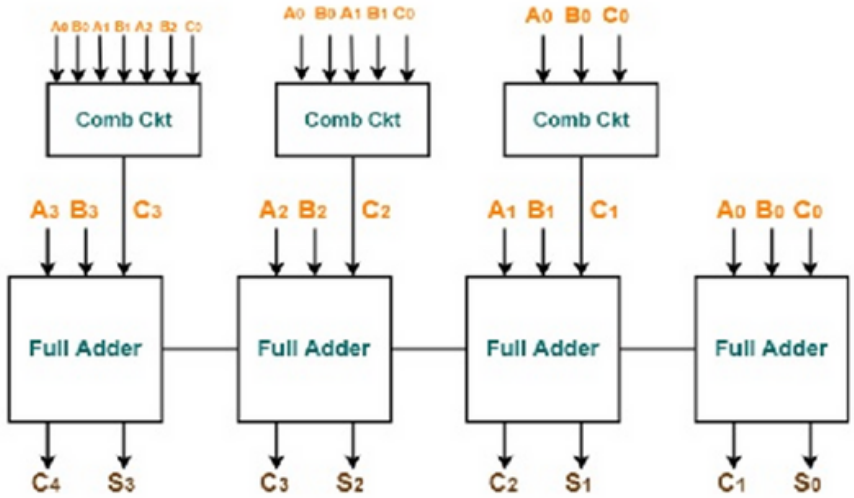
The adder produce carry propagation delay while performing other arithmetic operations like multiplication and divisions as it uses several additions or subtraction steps. This is a major problem for the adder and hence improving the speed of addition will improve the speed of all other arithmetic operations. Hence reducing the carry propagation delay of adders is of great importance. There are different logic design approaches that have been employed to overcome the carry propagation problem.

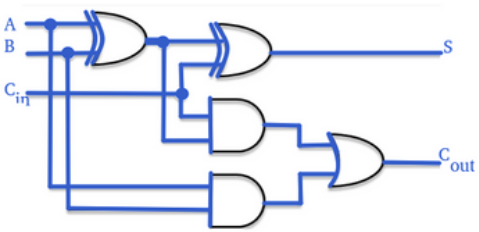
One widely used approach is to employ a carry look-ahead which solves this problem by calculating the carry signals in advance, based on the input signals. This type of adder circuit is called a carry look-ahead adder.

Here a carry signal will be generated in two cases:

- Input bits A and B are 1
- When one of the two bits is 1 and the carry-in is 1.

a 4-bit Carry Look-ahead Adder described below:





A	B	C	C +1	Condition
0	0	0	0	
0	0	1	0	No Carry
0	1	0	0	Generate
0	1	1	1	
1	0	0	0	No Carry
1	0	1	1	Propogate
1	1	0	1	Carry
1	1	1	1	Generate

Circuit design and Truth Table

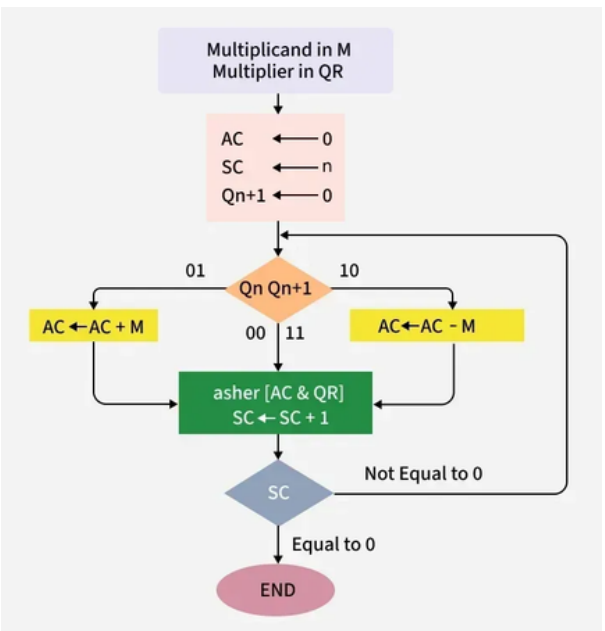
Booth's Algorithm

Booth's algorithm is a method for multiplying signed binary numbers in two's complement representation. It improves efficiency by minimizing the number of required arithmetic operations.

- The method works by examining pairs of adjacent bits in the multiplier and deciding whether to add, subtract, or do nothing with the multiplicand, followed by an arithmetic shift.
- This approach simplifies handling of consecutive sequences of 1s or 0s, making multiplication faster and more effective than the standard binary method.

Booth's Algorithm Flowchart

We name the registers as A, B and Q, AC, BR and QR, respectively. Qn designates the least significant bit of the multiplier in the register QR. An extra flip-flop Qn+1is appended to QR to facilitate a double inspection of the multiplier.



Booth's Algorithm Steps

- Initialize AC = 0, Qn+1 = 0, and set SC = n (number of multiplier bits).
- Check the two least significant bits (Qn and Qn+1).
- If bits = 10 → Subtract multiplicand (M) from AC (first 1 in a sequence).
- If bits = 01 → Add multiplicand (M) to AC (first 0 after ones).
- If bits = 00 or 11 → No change in AC (partial product unchanged).
- Perform an Arithmetic Shift Right (ashr) on [AC, QR, Qn+1] (sign bit in AC remains).
- Decrement sequence counter (SC).
- Repeat steps until SC = 0 (n iterations complete).
- Handle negative numbers: if multiplicand/multiplier is negative, take its 2's complement to simplify operations.
- Final product obtained in [AC, QR] after all steps

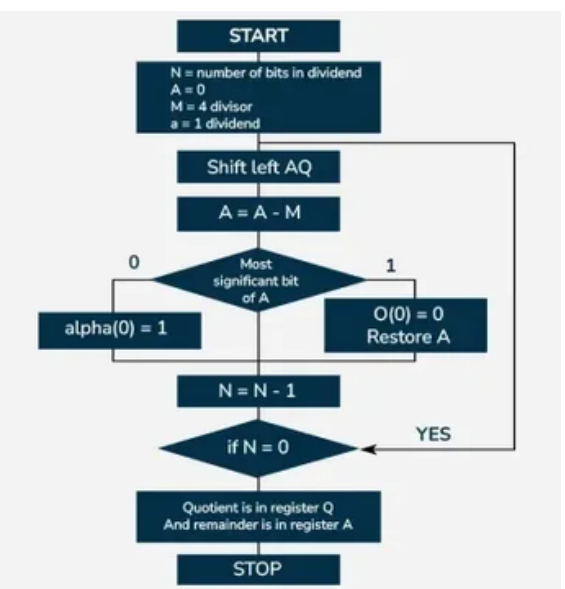
Example - A numerical example of booth's algorithm is shown below for n = 4. It shows the step by step multiplication of -5 and -7.

- BR = -5 = 1011,
- BR' = 0100, <-- 1's Complement (change the values 0 to 1 and 1 to 0)
- BR'+1 = 0101 <-- 2's Complement (add 1 to the Binary value obtained after 1's complement)
- QR = -7 = 1001 <-- 2's Complement of 0111 (7 = 0111 in Binary)

- The explanation of first step is as follows: Qn+1
- AC = 0000, QR = 1001, Qn+1 = 0, SC = 4
- Qn Qn+1 = 10
- So, we do AC + (BR)'+1, which gives AC = 0101
- On right shifting AC and QR, we get
- AC = 0010, QR = 1100 and Qn+1 = 1
- Product is calculated as follows:
- Product = AC QR
- Product = 0010 0011 = 35

Division Algorithms

- The Restoring Division Algorithm is a method for dividing two unsigned integers in binary form, producing a quotient and remainder through iterative shifting and subtraction.
- It uses registers for the quotient (Q), remainder (A), and divisor (M).
- If subtraction gives a negative result, the remainder is restored to its previous value, and the quotient bit is set to zero, hence the term "restoring."



Steps for Restoring Division Algorithm

- Step-1: First the registers are initialized with corresponding values (Q = Dividend, M = Divisor, A = 0, n = number of bits in dividend)
- Step-2: Then the content of register A and Q is shifted left as if they are a single unit
- Step-3: Then content of register M is subtracted from A and result is stored in A
- Step-4: Then the most significant bit of the A is checked if it is 0 the least significant bit of Q is set to 1 otherwise if it is 1 the least significant bit of Q is set to 0 and value of register A is restored i.e the value of A before the subtraction with M
- Step-5: The value of counter n is decremented
- Step-6: If the value of n becomes zero we get of the loop otherwise we repeat from step 2
- Step-7: Finally, the register Q contain the quotient and A contain remainder

Unsigned Integer

Unsigned integers store only non-negative numbers. Signed integers use the first bit for the sign (0 = positive, 1 = negative), while unsigned use all bits for value. In 8-bit form, unsigned integers range from 0 to 255. They are used in computing when only positive values or a larger range is required.

Example: Perform Division Restoring Algorithm
Dividend = 11 Divisor = 3

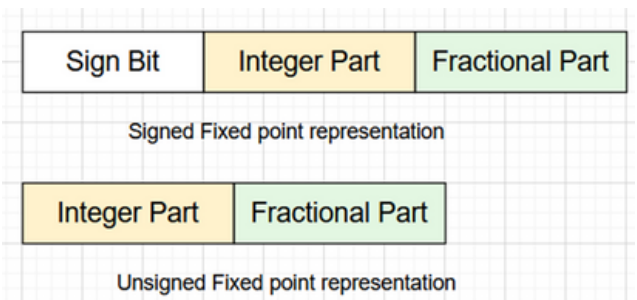
n	M	A	Q	Operati
4	11	0	1011	initialize
	11	1	011_	shift left
	11	11110	011_	A=A-M
	11	1	110	Q[0]=0
3	11	10	110_	shift left
	11	11111	110_	A=A-M
	11	10	1100	Q[0]=0
2	11	101	100_	shift left
	11	10	100_	A=A-M
	11	10	1001	Q[0]=1
1	11	101	001_	shift left
	11	10	001_	A=A-M
	11	10	11	Q[0]=1

Remember to restore the value of A most significant bit of A is 1. As that register Q contain the quotient, i.e. 3 and register A contain remainder 2.

FIXED POINT REPRESENTATION

- In computers, fixed-point representation is a real data type for numbers. Fixed point representation can convert data into binary form, and then the data is processed, stored, and used by the computer. It has a fixed number of bits for the integral and fractional parts. For example, if given fixed-point representation is IIIII.FFF, we can store a minimum value of 00000.001 and a maximum value of 99999.999.

- There are three parts of the fixed-point number representation: Sign bit, Integral part, and Fractional part. The below figure depicts it.



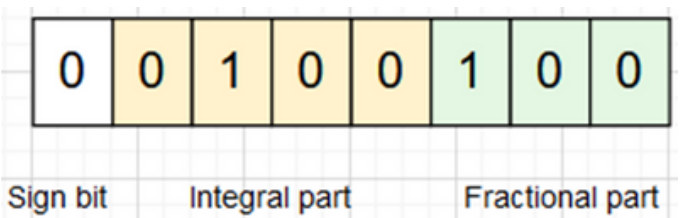
- **Sign bit**:- The fixed-point number representation in binary uses a sign bit. The negative number has a sign bit 1, while a positive number has a bit 0.
- **Integral Part**:- The integral part in fixed-point numbers is of different lengths at different places. It depends on the register's size; for an 8-bit register, the integral part is 4 bits.
- **Fractional part:**- The Fractional part is of different lengths at different places. It depends on the registers; for an 8-bit register, the fractional part is 3 bits.

Size of Sign Bit, Integer Part, and Fractional Part for different registers are displayed below:

Register	Sign Bit	Integer Part	Fraction Part
8-bit register	1 bit	4 bits	3 bits
16-bit register	1 bit	9 bits	6 bits
32-bit register	1 bit	15 bits	9 bits

Now that we have learned about fixed-point number representation, let's see how to represent it.
The number considered is 4.5

- Step 1: We will convert the number 4.5 to binary form.
  4.5 = 100.1
- Step 2: Represent the binary number in fixed-point notation with the following format.



Floating-Point Representation

In the floating-point Representation, we represent numbers as sign-mantissa and exponent form {S, M, E}. The floating-point Representation follows the IEEE 754 standard. Here is a figure which shows floating-point representation.

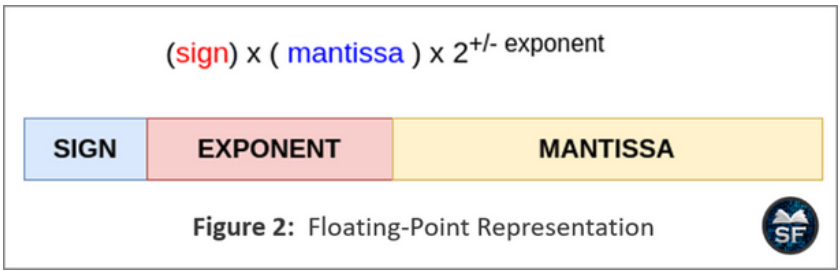


Figure 2: Floating-Point Representation

- The number of bits for its integer or fractional part is not fixed as its radix point is not fixed.
- It fixes the number of bits for the mantissa, sign, and exponent.
- The mantissa part gives the fractional part and the exponent gives the weight of the number in powers of 2.
- The sign tells whether the number is positive or negative. If it is 0 the number is positive and if 1, it is negative.

We have two precisions in floating-point: –

- Single-bit Precision: – 32-bits Representation, 1 sign bit, 23-bit mantissa, and 8-bit exponent
- Double-bit Precision: – 64-bits Representation, 1 sign bit, 52-bit mantissa, and 11-bit exponent

Steps to Represent a Number in Floating-Point

- Represent the number in its signed Binary form.
- The radix-point is moved to the 1st point, and then we represent it as number x 2 to the power (the number of places we shifted the decimal by to the first place.)

For example, 110.111 is written as 1.10111 x 2^2, as we shifted the decimal by two places.

- If the first bit is 1, it is called normalized significand. We do not represent the first bit in floating-point representation. It is always considered as a default value.
- To the power of 2, we add the number 127 as a bias in the case of 32-bit precision and we add 1023 as bias in the case of 64-bit precision.
- The value of exponent always has to lie between 1 to 254, as we do not take all zeros and all ones.
- We represent this biased power as exponent in its 8-bit binary form.
- The numbers after the decimal point are stored as mantissa and zeros are added at the end of the number if 23 bits aren't there.
- The sign bit is taken according to the negative or positive of the number.
-

For example, 1001101011.111 can be written as 1.001101011111 x 2^9, as we had to shift 9 decimal places to the left. After adding the bias of 127, we get 9 + 127 = 136. 136 in decimal is 10001000.

Thus in floating-point, it becomes,

0 10001000 001101011111

UNIT 3
Basic Processing Module

The "basic processing module" is the Central Processing Unit (CPU), which acts as the brain of the computer. Its fundamental job is to execute instructions.

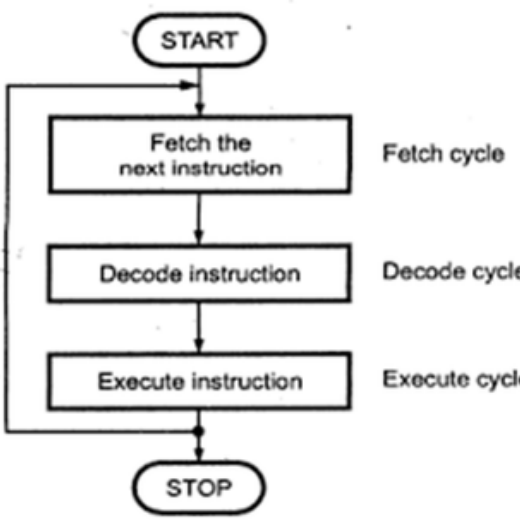
Fundamental concepts

Core Components of the CPU

1. **Arithmetic Logic Unit (ALU)**
   - What it is: The digital calculator of the CPU.
   - What it does:
     - Performs all mathematical calculations (addition , subtraction etc.).
     - Performs all logical operations (like AND, OR, NOT).
     - Performs comparisons (e.g., is value A greater than value B? $A > B$).
2. **Control Unit (CU)**
   - What it is: The manager or director of the CPU.
   - What it does:
     - It doesn't perform calculations itself.
     - It directs the flow of data and instructions.
     - It fetches instructions from memory, decodes them, and tells the other parts (like the ALU) what to do.
3. **Registers**
   - What it is: Small, extremely fast storage locations located directly inside the CPU.
   - What they do:
     - Temporarily hold the data and instructions that the CPU is currently working on.

Execution of a Complete Instruction (Fetch-Decode-Execute Cycle)

- A program residing in the memory unit of the computer consists of a sequence of instructions. The program is executed in the computer by going through a cycle for each instruction. Each instruction cycle in turn is subdivided into sequence of subcycles or phases
- The instruction cycle comprises three main stages and is also addressed as the fetch-decode-execute cycle or fetch-execute cycle because of the steps involved.
- They are as follows:
- 1. Fetch stage – Fetch the next instruction from the memory
- 2. Decode stage – Decode the instruction, reads the effective address from memory if the instruction has an indirect address.
- 3. Execute stage.- execute the instruction



Steps Involved in Instruction Fetch, Decode and Execute phase

Let's take a look at the steps involved in the instruction cycle from the beginning to the end:

Step 1: Fetch Phase – The first phase of the instruction cycle

- Here, the sequence counter (SC) is set to zero at the start of the instruction cycle.
- SC – 0

Step 2: Fetch phase occurring at Clock Pulse (T-0)

- Here, the instruction cycle stores the address of the next instruction in the program counter (PC) register. The PC's content (address) is assigned to the address register in the first clock cycle (T-0) (AR).
- AR<- PC

Step 3: Fetch phase occurring at Clock Pulse 1

- In the next clock cycle (T-1), the instruction is read from memory and loaded into the instruction register (IR), while the program counter (PC) is also incremented by one. The program counter (PC) now points to the memory address of the next instruction to be retrieved.
- PC <– PC+1 AND IR <-M [AR]

The Instruction Register (IR) is a 16-bit register capable of storing instructions in 16-bit format. The three components of the 16-bit instruction format (0 to 11 bit) are:

- 1.Addressing Mode (15th bit)
- 2.OPCODE (12, 13, 14 bit)
- 3.Operand Address

Step 4: Decode Phase occurring at Clock Pulse (T-2)

- The CPU's control unit decodes the program instruction once it is loaded into the instruction register (IR). A Decode Phase is the second phase of the instruction cycle.
- The instruction is decoded by the CPU's control unit based on the operation code (OPCODE), determined by three bits (bit 12, 13, 14).

Step 5: Decode Instruction Type occurring at Clock Pulse (T-2)

- The CPU's control unit must first decode the type of instruction. The three types of references instructions are:
- 1.Memory reference instruction
- 2.Register reference instruction
- 3.Input/output instruction.
- The 3 X 8 decoder determines the type of instruction. The D7 value determines the type of instruction. The three-bit OPCODE can have eight distinct values, starting with (DO, D1, D2, ..., D6, D7). Alternatively, the OPCODE can have values ranging from DO – 000 to D7 111.

Step 6: Decode Addressing Mode occurring at Clock Pulse (T-3)

- If D7 is set to zero, the instruction type is a memory reference. When D7 is set to 1, the instruction type can be a register reference type or an input/output instruction. The instruction type is register reference type if D7 equals one and I equals 0. If D7 and I are both 1, the instruction type is input/output.
- After deciding on the type of instructions and making a subsequent decision, the decode phase concludes. The value of I in the 15th bit determines the decode phase.

Step 7: Operand Address Calculation – The address of the operand is evaluated if it has a reference to an operand in memory or is applicable through the Input/Output.
Step 8: Operand Fetch – The operand is read from the memory or the I/O.
Step 9: Data Operation – The actual operation that the instruction contains is executed.
Step 10: Store Operands – It can store the result acquired in the memory or transfer it to the I/O.

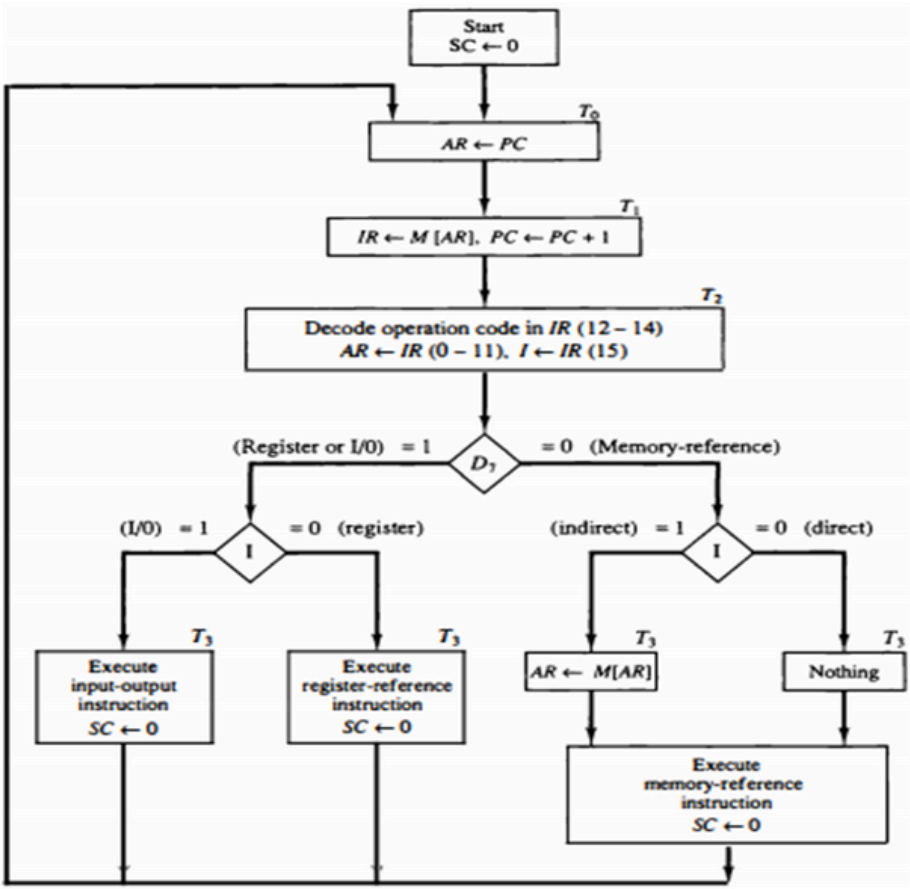


Figure Flowchart for instruction cycle (initial configuration).

Multiple Bus Organization

- Multiple bus organization in computer architecture is a design that allows multiple devices to work simultaneously. This reduces the time spent waiting and improves the computer's speed. The main advantage of multiple bus organization is the reduction in the number of cycles required for execution.
- In a multiple bus structure, one bus is used to fetch instructions and the other is used to fetch data. The same bus is shared by three units: memory, processor, and I/O units.
- In a multiple bus system, each processor-memory pair is linked by various redundant paths. This means that the failure of one or more paths can be tolerated, but it will degrade system performance.

The main reason for having multiple buses in a computer design is to improve performance. Other advantages include:

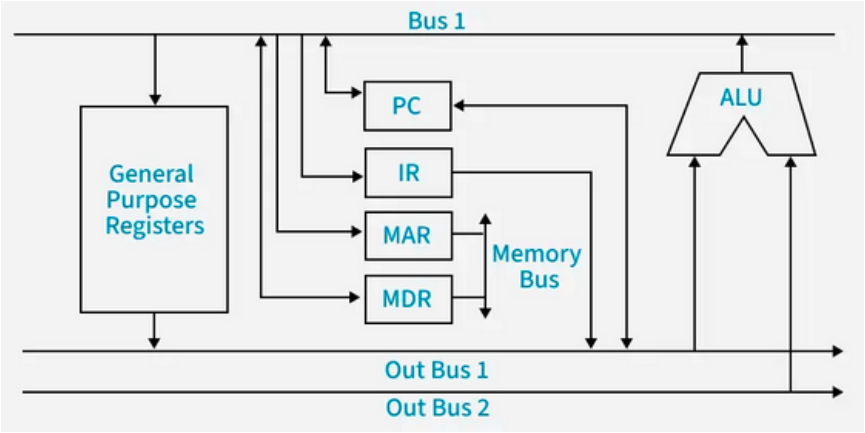
• Better connectivity

• An increase in the size of the registers

There are three types of bus lines: data bus, address bus, and control bus. Communication over each bus line is performed in cooperation with another.

Three bus organization of data path

In a three bus organization, there are OUT bus1, OUT bus2, and an IN bus. Operands from general-purpose registers flow through the OUT buses to the ALU, and the output is placed on the IN bus to the respective registers. It is more complex but faster, allowing parallel operand processing and immediate output transfer, overcoming the busy-waiting problem of two-bus systems.

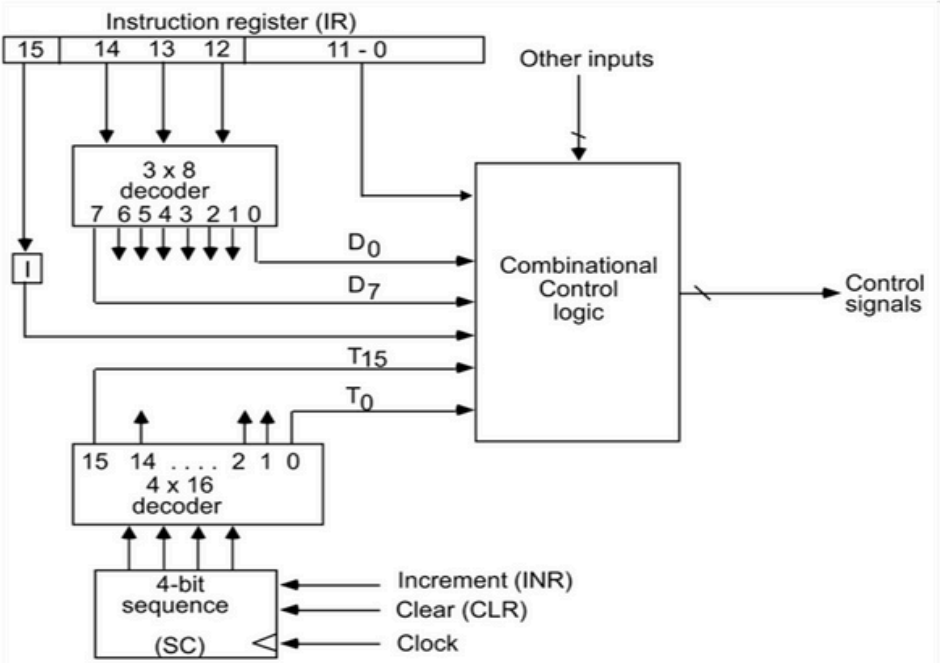


The main advantages of multiple bus organizations over the single bus are :

1. Increase in size of the registers.
2. Reduction in the number of cycles for execution.
3. Increases the speed of execution or we can say faster execution

CONTROL UNIT

The unit designed to produce control signals through control logic is called as Control unit. The Control unit can be hardwired or microprogrammed.



Control unit of basic computer (Hardwired Control Unit)

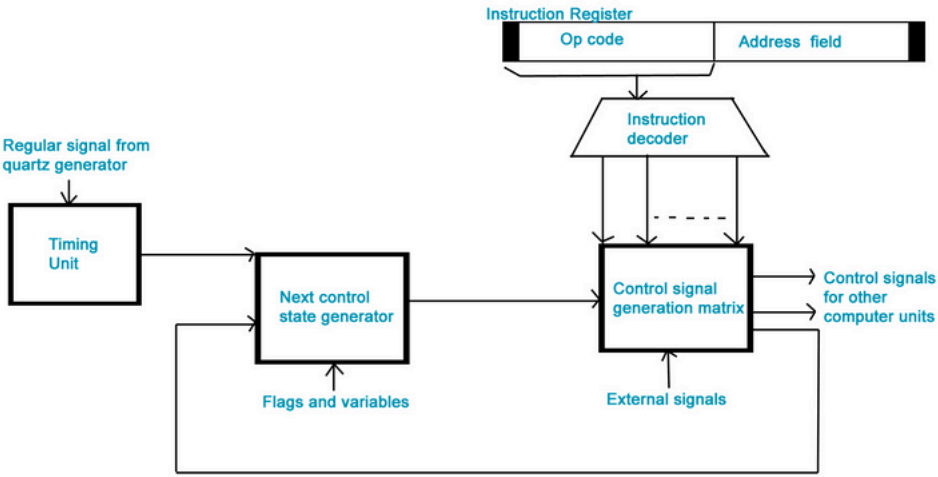
Components of Control unit are:

1. Two decoders
2. A sequence counter
3. Control logic gates

Hardwired Control Unit

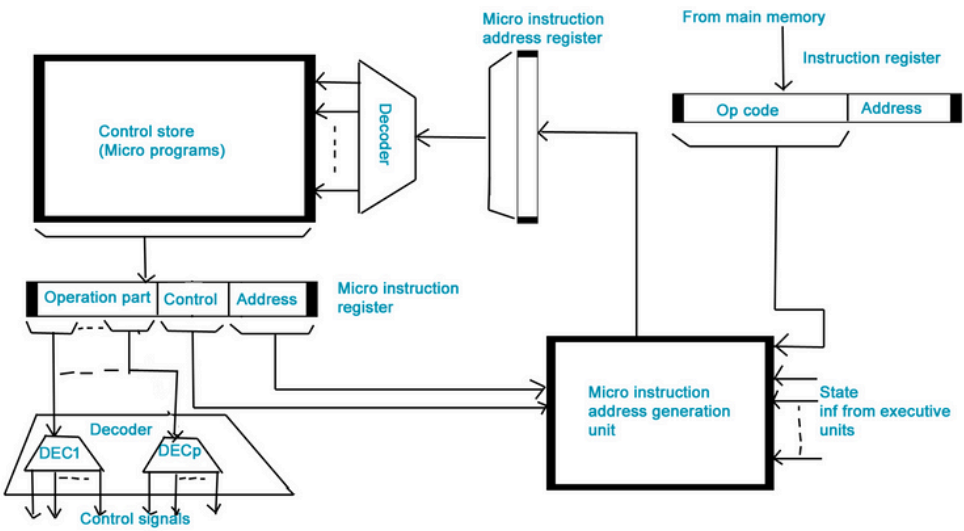
The control hardware can be viewed as a state machine that changes from one state to another in every clock cycle, depending on the contents of the instruction register, the condition codes, and the external inputs. The outputs of the state machine are the control signals. The sequence of the operation carried out by this machine is determined by the wiring of the logic elements and hence is named “hardwired”.

- Fixed logic circuits that correspond directly to the Boolean expressions are used to generate the control signals.
- Hardwired control is faster than microprogrammed control.
- A controller that uses this approach can operate at high speed.
- RISC architecture is based on the hardwired control unit



Micro-programmed Control Unit

- The control signals associated with operations are stored in special memory units inaccessible by the programmer as Control Words.
- Control signals are generated by a program that is similar to machine language programs.
- The micro-programmed control unit is slower in speed because of the time it takes to fetch microinstructions from the control memory.



Some important points

- Control Word: A control word is a word whose individual bits represent various control signals.
- Micro-routine: A sequence of control words corresponding to the control sequence of a machine instruction constitutes the micro-routine for that instruction.
- Micro-instruction: Individual control words in this micro-routine are referred to as microinstructions.
- Micro-program: A sequence of micro-instructions is called a micro-program, which is stored in a ROM or RAM called a Control Memory (CM).
- Control Store: the micro-routines for all instructions in the instruction set of a computer are stored in a special memory called the Control Store.

Pipelining And Hazards

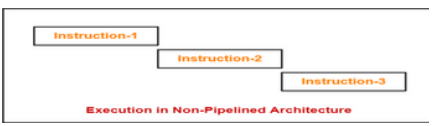
Pipelining is a technique for breaking down a sequential process into various sub-operations and executing each sub-operation in its own dedicated segment that runs in parallel with all other segments.
The most significant feature of a pipeline technique is that it allows several computations to run in parallel in different parts at the same time.

A program consists of several number of instructions.These instructions may be executed in the following two ways-

Non-Pipelined Execution-

In non-pipelined architecture,
·All the instructions of a program are executed sequentially one after the other.
·A new instruction executes only after the previous instruction has executed completely.
·This style of executing the instructions is highly inefficient.

Example-
Consider a program consisting of three instructions.
In a non-pipelined architecture, these instructions execute one after the other as-



Pipelined Execution-

In pipelined architecture,

·Multiple instructions are executed parallelly.

·This style of executing the instructions is highly efficient.

Pipelined Architecture-

In pipelined architecture,

·The hardware of the CPU is split up into several functional units.

·Each functional unit performs a dedicated task.

·The number of functional units may vary from processor to processor.

·These functional units are called as stages of the pipeline.

·Control unit manages all the stages using control signals.

·There is a register associated with each stage that holds the data.

·There is a global clock that synchronizes the working of all the stages.

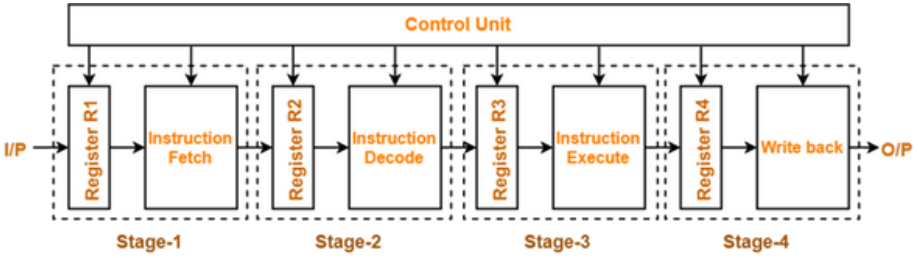
·At the beginning of each clock cycle, each stage takes the input from its register.

·Each stage then processes the data and feed its output to the register of the next stage.

Four-Stage Pipeline-

In four stage pipelined architecture, the execution of each instruction is completed in following 4 stages-

1. Instruction fetch (IF)
2. Instruction decode (ID)
3. Instruction Execute (IE)
4. Write back (WB)



Four Stage Instruction Pipeline

At stage-01,
·First functional unit performs instruction fetch.
·It fetches the instruction to be executed.
At stage-02,
·Second functional unit performs instruction decode.
·It decodes the instruction to be executed.
At stage-03,
·Third functional unit performs instruction execution.
·It executes the instruction.
At stage-04,
·Fourth functional unit performs write back.
It writes back the result so obtained after executing the instruction.

Types of Pipeline Hazards in Computer Architecture

The three different types of hazards in computer architecture are:
1. Structural
2. Data
3. Control
Dependencies can be addressed in a variety of ways. The easiest is to introduce a bubble into the pipeline, which stalls it and limits throughput. The bubble forces the next instruction to wait until the previous one is completed.

Structural Hazard

Hardware resource conflicts among the instructions in the pipeline cause structural hazards. Memory, a GPR Register, or an ALU might all be used as resources here. When more than one instruction in the pipe requires access to the very same resource in the same clock cycle, a resource conflict is said to arise. In an overlapping pipelined execution, this is a circumstance where the hardware cannot handle all potential combinations. Know more about structural hazards here.

Data Hazards

Data hazards in pipelining emerge when the execution of one instruction is dependent on the results of another instruction that is still being processed in the pipeline. The order of the READ or WRITE operations on the register is used to classify data threats into three groups. Know more about data hazards here.

Control Hazards

Branch hazards are caused by branch instructions and are known as control hazards in computer architecture. The flow of program/instruction execution is controlled by branch instructions. Remember that conditional statements are used in higher-level languages for iterative loops and condition testing (correlate with while, for, and if case statements). These are converted into one of the BRANCH instruction variations. As a result, when the decision to execute one instruction is reliant on the result of another instruction, such as a conditional branch

For E-BOOK:

KINDLY REFER TO THIS LINK FOR FLIPPING:

https://heyzine.com/flip-book/3e5340065c.html